TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

LAB 02

ỨNG DỤNG CỦA MẠCH SỐ

Họ và tên	Nguyễn Thanh Hoàng Lâm	**MSSV**	24520935
Lớp	IT012.R11.2	**Ngày thực hành**	12/10/2026
Tự chấm	10,0đ	**Link git**	IT012.R11.2-24520935-BaoCaoLab

1. Tổng quan buổi thực hành

Sinh viên dùng phần mềm **Logisim** để biến kiến thức Chương 4 (mạch số, mạch tổ hợp, mạch tuần tự, latch, flip-flop, thanh ghi) và Chương 5 (bộ cộng, ALU, bộ chọn, bộ so sánh, bộ giải mã, tập thanh ghi) thành một bài toán xuyên suốt: **tự xây một datapath 8 bit của CPU thu nhỏ**, từng khối một, theo chu trình **dự đoán – dựng mạch – quan sát – giải thích**.

Sau buổi thực hành, Sinh viên có thể:

- Thiết kế và dựng bộ cộng/trừ từ cổng luận lý theo quy trình 4 bước (bảng chân trị → biểu thức → tối ưu → sơ đồ), đọc được các **cờ** Z, N, C, V và phát hiện **tràn số**.
- Dựng **ALU 8 bit** từ bộ chọn (MUX), bộ cộng/trừ và cổng luận lý; dùng ALU để so sánh hai số và chỉ ra **giới hạn** của quy tắc so sánh bằng phép trừ.
- Phân biệt **latch** (tích cực theo mức) với **flip-flop** (tích cực theo cạnh) bằng dạng sóng; tự dựng D latch và ghép hai latch thành D flip-flop.
- Giải thích vì sao thanh ghi dùng flip-flop và vì sao vòng hồi tiếp "thanh ghi → ALU → thanh ghi" vẫn hoạt động đúng.
- Dựng **tập thanh ghi 4 × 8 bit**, ghép với ALU thành đường dữ liệu và điều khiển bằng tay để chạy một chương trình ngắn (C = A + B, hoán đổi hai thanh ghi).

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Làm quen bus nhiều bit, mạch con, clock; kiểm tra môi trường	15 phút	
TN 1	Bộ cộng/trừ 4 bit	45 phút	2,0
TN 2	ALU 8 bit và bộ so sánh	45 phút	1,5
TN 3	Mạch tuần tự: D latch, D flip-flop	45 phút	3,0
TN 4	Tập thanh ghi và đường dữ liệu 8 bit	60 phút	3,0
Kết	Tổng kết, phản tư, nộp bài	15 phút	0,5
Tổng		**225 phút**	**10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ Sinh viên điền hoặc trả lời trực tiếp vào file này (gõ chữ, hoặc dán ảnh chụp màn hình vào khung nét đứt).
- Ký hiệu: NOT là dấu phẩy trên (A'); AND viết liền hoặc A·B; OR là A + B; XOR là A ⊕ B. Số thập lục phân viết 0x3C. Với bus nhiều bit, A[7] là bit trọng số cao nhất (MSB). Số có dấu dùng biểu diễn **bù 2** (Chương 2).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Ghi dự đoán **trước** khi mô phỏng; dự đoán sai **không bị trừ điểm**, phần giải thích vì sao sai mới được chấm.
- Các mạch con (subcircuit) tạo ở thí nghiệm trước được **dùng lại** ở thí nghiệm sau. Đặt tên đúng quy ước và không xoá.

PHẦN 0 — Làm quen bus, mạch con và clock (15 phút)

Sinh viên đã dùng Logisim ở Lab 01; phần này chỉ bổ sung các kỹ năng mới cho mạch nhiều bit và mạch tuần tự. Giảng viên hướng dẫn nhanh theo tài liệu *Hướng dẫn sử dụng Logisim*. Sinh viên tự đánh dấu ☑ khi thực hiện được từng kỹ năng (bỏ qua nếu đã thành thạo).

☑	Kỹ năng	Gợi ý
☑	Tạo project mới; đặt tên circuit đầu tiên là **Warmup**; lưu file **IT012_Lab02_<MSSV>.circ**	Mỗi thí nghiệm dùng các circuit riêng trong **cùng một file** (Project ▸ Add Circuit); tên circuit theo quy ước ở đầu mỗi thí nghiệm.
☑	Đặt Input Pin / Output Pin **nhiều bit** (thuộc tính Data Bits); đổi từng bit bằng Poke tool	Pin 8 bit hiển thị 8 ô bit. Dùng **Probe** (thư viện Wiring) và đổi thuộc tính **Radix** để đọc ở dạng nhị phân, thập lục phân, thập phân có dấu.
☑	Dùng **Splitter** để tách bus thành từng dây và ghép các dây thành bus	Fan Out = số dây, Bit Width In = độ rộng bus. Kiểm tra thứ tự bit (bit 0 là LSB).
☑	Tạo **mạch con**: dựng một circuit, rồi kéo circuit đó từ ngăn Explorer vào circuit khác như một linh kiện	Vị trí các pin trong circuit quyết định vị trí các chân của linh kiện; đặt nhãn pin rõ ràng.
☑	Dùng linh kiện có sẵn: **Plexers** (Multiplexer, Decoder), **Arithmetic** (Adder, Subtractor), **Memory** (D Flip-Flop, Register)	Chọn linh kiện rồi chỉnh Data Bits, Select Bits ở ngăn thuộc tính (góc dưới bên trái).
☑	Điều khiển mạch tuần tự: đổi tay một Input Pin **CLK**; dùng **Simulate ▸ Reset Simulation** để đưa mạch về trạng thái ban đầu	Cạnh lên xảy ra khi Poke CLK từ 0 lên 1. Với linh kiện Clock (Wiring), mỗi lần Simulate ▸ Tick Once đổi Clock một nửa chu kỳ.
☑	(Tuỳ chọn) Xem dạng sóng: **Chronogram** (Logisim-evolution) hoặc cửa sổ **Simulate ▸ Logging** (Logisim 2.7.x)	Tuỳ phiên bản; không bắt buộc, dùng để kiểm tra lại Bảng 10 ở TN 3.
☑	Chụp màn hình mạch và dán vào khung ảnh của file này	Chụp cả mạch lẫn nhãn ngõ vào/ngõ ra và giá trị đang hiển thị.

Phiên bản Logisim đang dùng	Logisim 2.7.1

Đọc màu dây và giá trị trong Logisim

Xanh lá sáng = 1 · **Xanh lá đậm** = 0 · **Xanh dương** = chưa xác định / chưa được điều khiển · **Đỏ** = lỗi/xung đột · **Cam** = hai đầu dây lệch số bit.

Ở mạch **tuần tự**, ngay sau khi mở file hoặc Reset Simulation, giá trị đang lưu có thể **chưa xác định** và hiển thị là **x** (hoặc dây xanh dương) cho đến khi mạch được ghi lần đầu. Đó là hành vi bình thường, không phải lỗi; nếu phiên bản Logisim của Sinh viên hiển thị khác, hãy ghi lại điều quan sát được.

Bước 0 – Kiểm tra nhanh. Trong circuit **Warmup**, đặt hai Input Pin 8 bit **A**, **B**, một **Adder** (Arithmetic, Data Bits = 8), một Output Pin 8 bit **S** và một Probe. **Dự đoán** S (viết hex) bằng tay trước khi Poke, rồi đọc S ở dạng thập phân **không dấu** và **có dấu** (bù 2).

A	B	S = A + B (hex)	S không dấu	S có dấu (bù 2)
0x20	0x0F	0x2F	47	47
0x7F	0x01	0x80	128	-128
0xFF	0x01	0x00	0	0
0x64	0x64	0xC8	200	-56

Circuit Warmup chỉ để làm quen, không nộp. Hãy để ý: cùng một dãy bit, hai cách đọc cho hai giá trị khác nhau; Thí nghiệm 1 sẽ trả lời "phần cứng báo cho phần mềm biết kết quả có đáng tin hay không" bằng cách nào.

THÍ NGHIỆM 1 — Bộ cộng/trừ 4 bit và các cờ trạng thái

Thời lượng gợi ý: 45 phút · Circuit: **FA, ADDSUB4** · Điểm: 2,0

Tình huống

Phép cộng là phép toán nền tảng của ALU: phép trừ, tăng/giảm và so sánh đều quy về bộ cộng. Sinh viên sẽ tự xây bộ cộng 4 bit từ cổng luận lý, biến nó thành bộ cộng/trừ, rồi tự thiết kế mạch **phát hiện tràn số** để biết khi nào kết quả 4 bit không còn đúng, thông tin mà CPU thật luôn cung cấp dưới dạng các cờ.

A. Bộ cộng toàn phần (FA) từ cổng luận lý

Bước 1.1 – Dự đoán. Cộng ba bit A + B + Cin cho kết quả từ 0 đến 3, viết được bằng hai bit (Cout S). Điền hai cột "dự đoán" của Bảng 1 bằng phép cộng số học, **chưa** dựng mạch.

DỰNG MẠCH **Bước 1.2.** Trong circuit **FA** (circuit mới), đặt 3 Input Pin **A, B, Cin** và 2 Output Pin **S, Cout**. Dựng theo slide: S = A ⊕ B ⊕ Cin; Cout = A·B + Cin·(A ⊕ B), dùng chung cổng A ⊕ B cho cả hai ngõ ra.

QUAN SÁT Dùng Poke tool (hoặc Analyze Circuit ▸ Table) để điền hai cột "đo".

Bảng 1. Bộ cộng toàn phần FA

A	B	Cin	S (dự đoán)	Cout (dự đoán)	S (đo)	Cout (đo)
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	1	0	1	0
0	1	1	0	1	0	1
1	0	0	1	0	1	0
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	1

Câu 1.1 (Phân tích). Cột Cout chính là hàm "có từ 2 ngõ vào trở lên bằng 1" mà slide Chương 4 dùng làm ví dụ thiết kế. Viết SoP tối giản của Cout (gom bìa Karnaugh 3 biến nếu cần) và tính C(B). So sánh với dạng Cout = A·B + Cin·(A ⊕ B) khi cổng A ⊕ B đã có sẵn để tạo S: tổng số cổng của cả mạch FA ở mỗi cách là bao nhiêu?

Trả lời: Biểu thức SoP tối giản: Cout = A . B + A . Cin + B . Cin . Biểu thức có 3 term, mỗi term gồm 2 literal → C(B) = 3 + 2 + 2 + 2 = 9. Nếu chỉ tính số cổng logic và cho phép OR ba ngõ vào, công thức SoP cần 3 AND và 1 OR, tức 4 cổng để tạo Cout. Cộng thêm 2 XOR để tạo S, toàn bộ FA cần 6 cổng. Còn công thức dùng chung XOR: Cout = A.B + Cin.(A ⊕ B) chỉ cần 2 XOR, 2 AND và 1 OR, tổng cộng 5 cổng cho cả FA.

Câu 1.2 (Phân tích). Đếm **số tầng cổng** (độ sâu) của mạch FA ở bước 1.2 trên hai đường đi: (a) từ **Cin** đến **Cout**; (b) từ **A hoặc B** đến **Cout**. Ghi cách đếm.

Trả lời: Từ Cin đến Cout: Cin → AND → OR có 2 tầng cổng.
Từ A hoặc B đến Cout trên đường dài nhất: A/B → XOR → AND → OR có 3 tầng cổng.
Độ sâu được đếm bằng số cổng mà tín hiệu phải đi qua trên đường từ đầu vào đến đầu ra, không tính Input Pin, Output Pin và dây nối

B. Ghép thành bộ cộng 4 bit nối tiếp

DỰNG MẠCH **Bước 1.3.** Trong circuit **ADDSUB4**: Input Pin **A** (4 bit), **B** (4 bit), **SUB** (1 bit); Output Pin **S** (4 bit), **C** (1 bit). **Lần này chưa dùng SUB:** nối 4 mạch con FA liên tiếp (Cout của tầng i vào Cin của tầng i + 1; Cin của tầng 0 nối hằng 0). Dùng Splitter để tách A, B và gộp S. Đặt Probe (hoặc Output Pin tạm) cho ba dây nhớ **C1, C2, C3** giữa các tầng.

DỰ ĐOÁN Với ba phép cộng ở Bảng 2, dự đoán S, các bit nhớ và **số tầng mà bit nhớ đã lan qua** (bắt đầu đếm từ khi C1 = 1).

Bảng 2. Bit nhớ lan truyền trong bộ cộng 4 bit (Cin = 0)

A	B	S (nhị phân)	C1	C2	C3	Cout	Số tầng bit nhớ lan qua
0001	0001	0010	1	0	0	0	1
0111	0001	1000	1	1	1	0	3
1111	0001	0000	1	1	1	1	4

Câu 1.3 (Đánh giá). Với bộ cộng nối tiếp n bit, bit nhớ có thể phải đi qua tất cả các tầng. Dựa vào Câu 1.2, điền số tầng cổng của đường đi dài nhất (từ A[0]/B[0] đến Cout) ở Bảng 3 và rút ra công thức theo n. Nhận xét: vì sao CPU 32/64 bit thực tế không dùng thuần bộ cộng nối tiếp?

Bảng 3. Độ sâu của bộ cộng nối tiếp

n (số bit)	Số tầng FA mà bit nhớ đi qua	Số tầng cổng đến Cout
4	4	9
8	8	17
32	32	65
n	n	2n + 1

Công thức, nhận xét: Ở Câu 1.2 ta đã biết: Từ A hoặc B đến Cout của FA đầu tiên có thể đi qua 3 tầng cổng và từ Cin đến Cout của mỗi FA tiếp theo đi qua 2 tầng cổng. Vậy công thức đường dài nhất: D(n)=3+2(n-1) = 2n+1.
Bộ cộng nối tiếp có độ trễ tăng tuyến tính theo số bit. Mỗi tầng phải đợi bit nhớ từ tầng trước, nên khi dùng bộ cộng 32 hoặc 64 bit, đường truyền bit nhớ có thể rất dài, làm chậm tốc độ xử lý. Vì vậy các CPU hiện đại thường dùng bộ cộng nhanh hơn như Carry Lookahead Adder hoặc Parallel Prefix Adder để giảm thời gian lan truyền bit nhớ.

C. Biến thành bộ cộng/trừ

Câu 1.4 (Hiểu). A − B có thể viết là A + (−B), mà theo Chương 2, −B = B' + 1 (bù 2). Từ đó giải thích vì sao chỉ cần thêm 4 cổng XOR (mỗi cổng nhận B[i] và SUB) và nối SUB vào Cin của tầng 0 là bộ cộng thành bộ cộng/trừ. Điều gì xảy ra ở hai trường hợp SUB = 0 và SUB = 1?

Trả lời: Theo phương pháp biểu diễn bù 2, phép trừ A − B được thực hiện bằng A + B' + 1.
Sử dụng 4 cổng XOR với tín hiệu SUB giúp lựa chọn giữa B và phần bù của B. Khi SUB = 0, đầu ra các cổng XOR giữ nguyên các bit của B và Cin tầng đầu bằng 0, nên mạch thực hiện phép cộng A + B. Khi SUB = 1, các cổng XOR đảo từng bit B và Cin tầng đầu bằng 1, nên mạch thực hiện A + B' + 1 = A − B.
Nhờ đó, cùng một bộ cộng 4 bit có thể thực hiện cả phép cộng và phép trừ mà không cần thiết kế riêng bộ trừ.

DỰNG MẠCH **Bước 1.4.** Thêm vào ADDSUB4: 4 cổng XOR cho B'[i] = B[i] ⊕ SUB; nối SUB vào Cin của tầng 0. Kiểm tra nhanh: SUB = 0, A = 0011, B = 0100 → S = 0111; SUB = 1, A = 0111, B = 0011 → S = 0100.

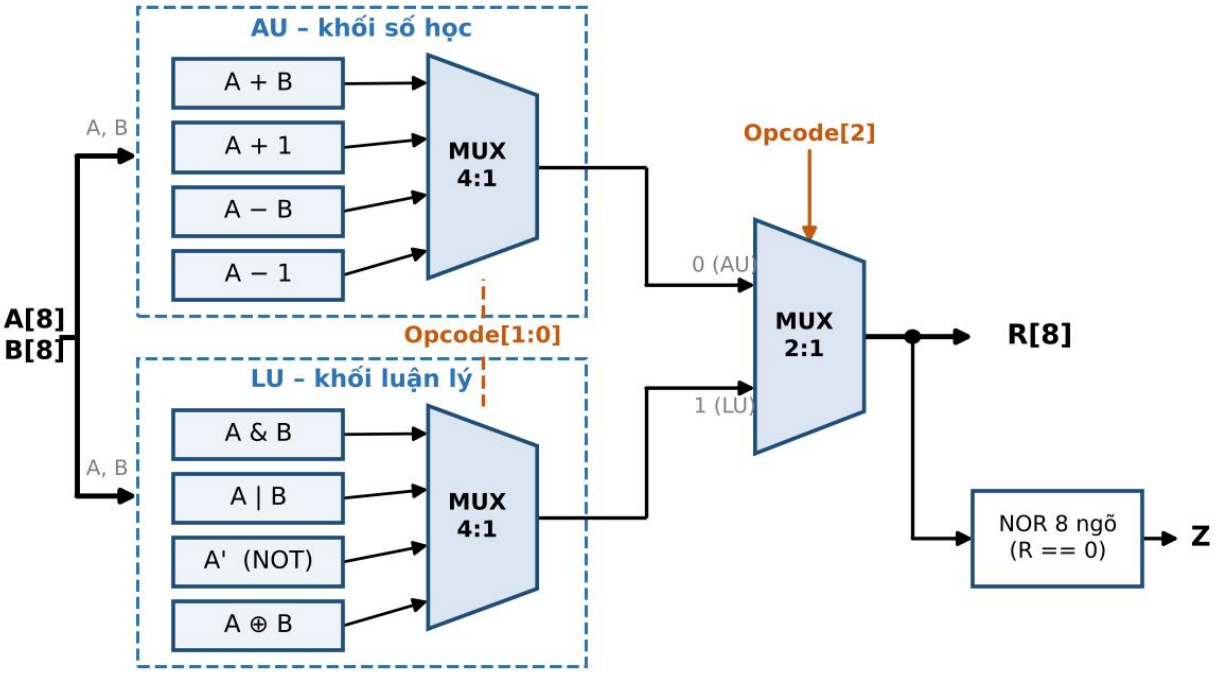
THÍ NGHIỆM 2 — ALU 8 bit và bộ so sánh: quy tắc nào luôn đúng?

Thời lượng gợi ý: 45 phút · Circuit: **ALU8, LT** · Điểm: 1,5

Tình huống

Một CPU chỉ cần **một** khối tính toán dùng chung cho nhiều phép toán: ALU. Sinh viên dựng ALU 8 bit theo bảng Opcode ở slide Chương 5 (Hình 1), rồi dùng chính ALU để **so sánh hai số** thay vì thiết kế bộ so sánh riêng, vì bảng chân trị của bộ so sánh hai số 8 bit có 2^16 hàng. Slide nêu quy tắc: "Y = A − B; Y[MSB] = 1 ⇒ A < B; Y[MSB] = 0 ⇒ A > B". Sinh viên sẽ **kiểm tra** quy tắc này có luôn đúng không.

A. Dựng ALU8 theo cấu trúc AU – LU – MUX



Hình 1. *Cấu trúc ALU 8 bit theo slide Chương 5. Opcode[1:0] chọn phép toán trong từng khối; Opcode[2] chọn khối AU hay LU.*

DỰNG MẠCH **Bước 2.1.** Trong circuit **ALU8**: Input Pin **A, B** (8 bit), **OP** (3 bit); Output Pin **R** (8 bit), **Z**. (a) **AU:** hai Adder (A + B, A + 1) và hai Subtractor (A − B, A − 1); hằng 1 lấy từ Constant 8 bit; chọn kết quả bằng Multiplexer 4:1 (8 bit) với Select = OP[1:0] (dùng Splitter tách OP). (b) **LU:** các cổng AND, OR, NOT, XOR có Data Bits = 8; chọn bằng một Multiplexer 4:1 khác với Select = OP[1:0]. (c) Multiplexer 2:1 (8 bit) chọn giữa AU và LU theo OP[2]. (d) **Z** = NOR của 8 bit R (Splitter + cổng NOR 8 ngõ vào).

Gợi ý: ngõ vào số k của Multiplexer được chọn khi Select = k. Hãy đối chiếu thứ tự nối với bảng Opcode ở slide (000: A + B, 001: A + 1, 010: A − B, 011: A − 1, 100: A & B, 101: A | B, 110: A', 111: A ⊕ B).

DỰ ĐOÁN Điền cột "dự đoán" của Bảng 6 bằng tay (hex, 8 bit) **trước khi** Poke.

QUAN SÁT **Ca 1:** A = 0x3C, B = 0x0F. **Ca 2:** A = 0x00, B = 0x01. Điền cột "đo" và cờ Z của Ca 2.

Bảng 6. Kiểm thử ALU8 với 8 Opcode

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
000	A + B	0x4B	0x4B	0x01	0x01	0

Opcode	Phép toán	Ca 1: R dự đoán	Ca 1: R đo	Ca 2: R dự đoán	Ca 2: R đo	Ca 2: Z
001	A + 1	0x3D	0x3D	0x01	0x01	0
010	A − B	0x2D	0x2D	0xFF	0xFF	0
011	A − 1	0x3B	0x3B	0xFF	0xFF	0
100	A & B	0x0C	0x0C	0x00	0x00	1
101	A | B	0x3F	0x3F	0x01	0x01	0
110	A'	0xC3	0xC3	0xFF	0xFF	0
111	A ⊕ B	0x33	0x33	0x01	0x01	0

Câu 2.1 (Phân tích). Ở Ca 2, phép nào cho R = 0xFF? Giá trị đó là 255 hay −1? ALU có "biết" điều đó không? Z = 1 ở phép nào, và Z = 1 có đồng nghĩa với "hai toán hạng bằng nhau" không?

Trả lời: Ở Ca 2, các Opcode cho R = 0xFF là: 010 (A−B), 011(A−1), 110 (NOT A). 0xFF có thể được hiểu là 255 nếu không dấu, hoặc −1 nếu có dấu bù 2. ALU không tự biết cách phần mềm sẽ diễn giải kết quả mà nó chỉ tạo ra một dãy 8 bit. Cờ Z bằng 1 chỉ ở Opcode 100 (AND), vì 0 AND 1 = 0. Do đó, Z = 1 không có nghĩa là A = B trong mọi phép toán. Chỉ khi đang thực hiện A−B thì Z = 1 mới cho biết hai toán hạng bằng nhau.

Câu 2.2 (Áp dụng). ALU này không có phép "chép" (MOV: R = A). Đề xuất **hai** cách dùng đúng ALU này để có R = A (ghi Opcode và giá trị cần đặt cho B). Cách nào dùng được khi ta **chỉ chọn được thanh ghi** cho A và B, không đặt được hằng số? (Sẽ dùng ở Thí nghiệm 4.)

Trả lời: Có thể sử dụng hai cách để thực hiện phép chép R=A bằng ALU:

- *Cách 1: Dùng phép OR (Opcode = 101), đặt B = 0x00. Khi đó: R=A OR 0=A.*
- *Cách 2: Dùng phép AND (Opcode = 100), đặt B = A. Khi đó: R=A AND A=A*

Khi chỉ được chọn thanh ghi cho A và B mà không thể đặt hằng số, cách 2 phù hợp hơn. Ta chỉ cần chọn cùng một thanh ghi cho cả hai đầu vào (RA = RB), khi đó ALU thực hiện A AND A=A, tương đương phép MOV mà không cần sử dụng hằng số

B. So sánh hai số bằng ALU

DỰ ĐOÁN Với 6 cặp (A, B) ở Bảng 7, tính R = A − B trên **8 bit**, rồi điền cột "Quy tắc slide kết luận" (dùng <, =, >). Điền hai cột "Thực tế" theo hai cách hiểu: A, B là số **có dấu** (bù 2) và là số **không dấu**.

QUAN SÁT Đặt Opcode = 010 trên ALU8, ghi R, Z, R[7] của từng cặp. Cột **LT đo** để trống, sẽ điền sau Bước 2.3.

Bảng 7. So sánh bằng phép trừ

A	B	R = A − B (đo)	Z	R[7]	Quy tắc slide kết luận	Thực tế (có dấu)	Thực tế (không dấu)	LT đo
0x05	0x07	0xFE	0	1	A<B	A<B	A<B	1
0x09	0x09	0x00	1	0	A=B	A=B	A=B	0
0x64	0x9C	0xC8	0	1	A<B	A>B	A<B	0
0x9C	0x64	0x38	0	0	A>B	A<B	A>B	1
0xC8	0x0A	0xBE	0	1	A<B	A<B	A>B	1
0x0A	0xC8	0x42	0	0	A>B	A>B	A<B	0

Câu 2.3 (Phân tích). Tìm các dòng mà quy tắc slide cho kết luận **sai**. Với mỗi dòng sai, nêu nguyên nhân (gợi ý: phép trừ 8 bit có bị tràn không? A và B đang được hiểu là số có dấu hay không dấu?). Quy tắc ở slide chỉ đáng tin trong điều kiện nào?

Trả lời: Với số có dấu, quy tắc dựa vào R[7] chỉ đáng tin khi phép trừ không gây tràn số có dấu. Đồng thời cần kiểm tra Z để phân biệt kết quả bằng nhau. Với số không dấu, bit R[7] không phải bit dấu nên không thể dùng làm quy tắc so sánh tổng quát. Cần dựa vào thông tin nhớ hoặc mượn từ phép trừ.

A	*B*	*Trường hợp sai*	*Nguyên nhân*
0x64	*0x9C*	*Số có dấu*	*Phép trừ gây tràn số có dấu*
0x9C	*0x64*	*Số có dấu*	*Phép trừ gây tràn số có dấu*
0xC8	*0x0A*	*Số không dấu*	*MSB = 1 không có nghĩa A nhỏ hơn B*
0x0A	*0xC8*	*Số không dấu*	*MSB = 0 không có nghĩa A lớn hơn B*

THIẾT KẾ **Bước 2.2 – Sửa quy tắc cho số có dấu.** Thiết kế mạch **LT** (A < B, số có dấu) chỉ dựa vào ba bit A[7], B[7], R[7] (R = A − B). Gợi ý: nếu A và B cùng dấu thì phép trừ không bao giờ tràn nên LT = R[7]; nếu khác dấu thì LT phụ thuộc dấu của A. Điền Bảng 8, bìa Karnaugh, rồi viết biểu thức tối giản.

Bảng 8. Bảng chân trị của LT(A7, B7, R7)

A7	B7	R7	LT (A < B có dấu)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1

A7	B7	R7	LT (A < B có dấu)
1	0	1	1
1	1	0	0
1	1	1	1

A7 \ B7 R7	00	01	11	10
0	0	1	0	0
1	1	1	1	0

LT tối giản = …	A7.B7’ + A7.R7 + B7’.R7
Số cổng cần dùng (kể cả cổng NOT)	5 cổng(1 not, 3 and, 1 or)

Câu 2.4 (Phân tích). Biểu thức LT quen thuộc không? (Gợi ý: so sánh với hàm đa số ở slide Chương 4 và với Cout của FA ở Thí nghiệm 1, trong đó một ngõ vào được lấy bù.) Giải thích bằng lời vì sao có sự giống nhau đó (nhớ lại A − B = A + B' + 1).

Trả lời: Biểu thức này có dạng giống hàm đa số (Majority Function) đã học ở Chương 4:M(X,Y,Z)=XY+XZ+YZ
Sự giống nhau xuất phát từ việc phép trừ được thực hiện bằng phép cộng bù 2:A-B=A+B’+1.
Do đó, mạch so sánh có dấu LT có cấu trúc logic tương tự hàm tạo bit nhớ của bộ cộng, với các biến A7,B7’ ,R7. Cả hai đều có dạng hàm đa số ba biến, tuy nhiên LT không phải chính là bit Cout của bộ trừ.

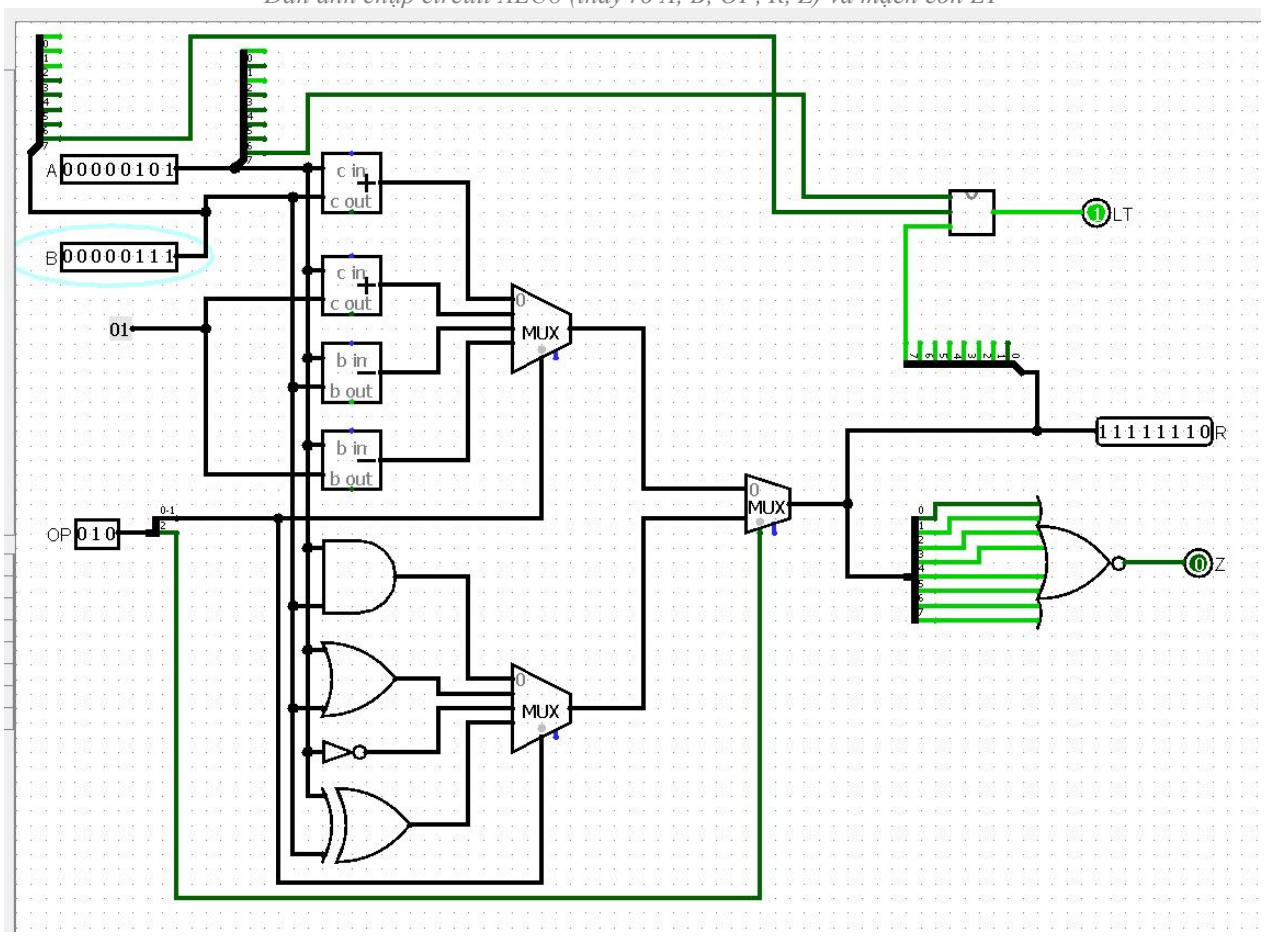
DỰNG MẠCH **Bước 2.3.** Dựng mạch con **LT** (ngõ vào A7, B7, R7; ngõ ra LT), nối vào ALU8 (A[7], B[7], R[7]) và điền cột **LT đo** của Bảng 7. LT có sửa được các dòng sai ở Câu 2.3 không?

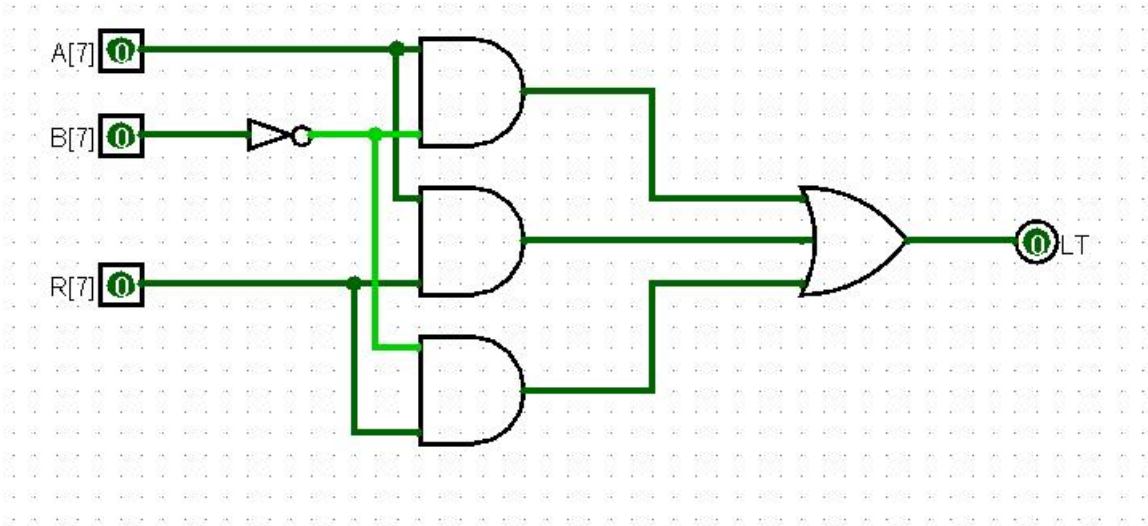
Câu 2.5 (Đánh giá – mở rộng). Với số **không dấu**, A < B khi nào? (Gợi ý: ý nghĩa của cờ C khi trừ ở Câu 1.6b.) Kiểm tra điều kiện của Sinh viên trên hai dòng 0xC8 và 0x0A của Bảng 7 bằng cách tính A − B trên 9 bit. Sau đó, so sánh chi phí: bộ so sánh trực tiếp hai số 8 bit cần bảng chân trị bao nhiêu hàng, so với “ALU + LT”? Khi nào vẫn nên dùng bộ so sánh riêng?

Trả lời: Với số không dấu, A < B khi phép trừ A − B cần mượn bit, tương ứng cờ C = 0 (C là carry-out khi trừ bằng phép cộng bù 2).
Kiểm tra bằng phép trừ 9 bit:
- *A = 0xC8 (200), B = 0x0A (10): A − B = 190 = 010111110 → Không mượn (C = 1), suy ra A > B.*
- *A= 0x0A (10), B = 0xC8 (200): A − B = −190 = 101000010 (bù 2 trên 9 bit) → Có mượn (C = 0), suy ra A < B.*

So sánh chi phí: Bộ so sánh trực tiếp hai số 8 bit có bảng chân trị \(2^{16}=65.536\) hàng.
Phương án ALU + LT tận dụng bộ trừ có sẵn và chỉ cần bổ sung logic so sánh, giúp tiết kiệm phần cứng.
Dùng bộ so sánh riêng khi cần so sánh độc lập hoặc song song với ALU, yêu cầu tốc độ cao hoặc cần tối ưu riêng cho chức năng so sánh

Dán ảnh chụp circuit ALU8 (thấy rõ A, B, OP, R, Z) và mạch con LT



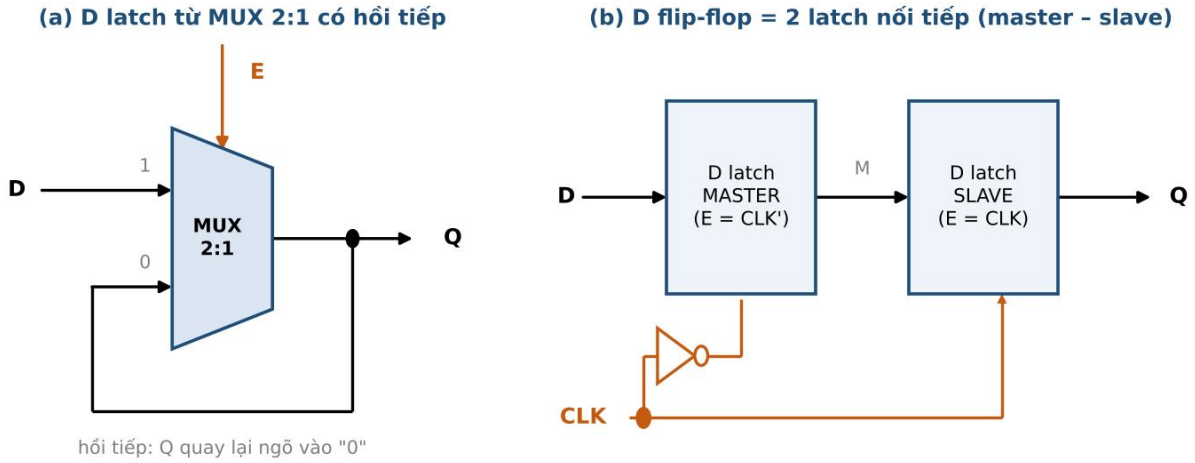


THÍ NGHIỆM 3 — Mạch tuần tự: D latch và D flip-flop

Thời lượng gợi ý: 45 phút · Circuit: **TN3 (mạch con: DLATCH, DFF_MS)** · Điểm: 3,0

Tình huống

ALU ở Thí nghiệm 2 là mạch tổ hợp: ngõ ra đổi ngay khi ngõ vào đổi và **không nhớ** gì. Muốn CPU giữ lại kết quả để dùng cho lệnh sau, cần **thiết bị lưu trữ** (slide Chương 4). Sinh viên sẽ tự dựng D latch từ bảng trạng thái ở slide, so sánh nó với D flip-flop có sẵn, rồi ghép hai latch thành một flip-flop và tìm hiểu vì sao thanh ghi phải dùng flip-flop.



Hình 2. *(a) D latch dựng từ MUX 2:1 có hồi tiếp; (b) D flip-flop ghép từ hai latch (master – slave).*

A. D latch dựng từ bảng trạng thái

Slide Chương 4 mô tả D latch bằng bảng trạng thái kế tiếp: **Q+ = Q khi E = 0** (giữ), **Q+ = D khi E = 1** (nhận). Đó chính là một **MUX 2:1** mà ngõ vào "giữ" được nối với **ngõ ra Q của chính nó** (Hình 2a).

DỰNG MẠCH **Bước 3.1.** Trong circuit **DLATCH**: Input Pin **D, E**; Output Pin **Q**; một Multiplexer 1 bit (Select = E, ngõ vào 1 = D, ngõ vào 0 = **Q nối ngược về**).

DỰ ĐOÁN Chọn **Simulate ▸ Reset Simulation**, rồi thực hiện lần lượt các thao tác ở Bảng 9 (đổi E và D bằng Poke). Điền Q dự đoán **trước**; nếu Q chưa xác định thì ghi **x**.

QUAN SÁT Sau đó điền cột "Q đo".

Bảng 9. Chuỗi thao tác trên DLATCH

Bước	E	D	Q dự đoán	Q đo
1	0	0	x	x
2	0	1	x	x
3	1	1	1	1
4	0	1	1	1
5	0	0	1	1
6	1	0	0	0
7	1	1	1	1
8	1	0	0	0

Bước	E	D	Q dự đoán	Q đo
9	0	0	0	0
10	0	1	0	0

Câu 3.1 (Hiểu). Vì sao ở bước 1–2 Q chưa xác định dù đã có D? Điều đó nói gì về việc phải **khởi tạo** (reset) thiết bị lưu trữ trong mạch thật?

Trả lời: Ở bước 1 và 2, tín hiệu E = 0 nên D latch ở trạng thái giữ giá trị Q trước đó. Tuy nhiên, sau khi Reset Simulation, mạch chưa được nạp dữ liệu hợp lệ nên trạng thái Q ban đầu có thể chưa xác định (x). Vì E = 0 nên dù D có giá trị 0 hay 1, Q vẫn không nhận dữ liệu mới.
Điều này cho thấy trong mạch thực tế, thiết bị lưu trữ cần được khởi tạo về trạng thái xác định trước khi sử dụng. Nếu không, trạng thái ban đầu không xác định có thể dẫn đến kết quả xử lý sai hoặc hoạt động không ổn định.

Câu 3.2 (Phân tích). Slide Chương 4 định nghĩa mạch tổ hợp là mạch **không có hồi tiếp**; DLATCH có đường hồi tiếp từ Q về MUX. Khi E = 0, mô tả đường đi của tín hiệu để giải thích vì sao hồi tiếp làm mạch "nhớ". Khi E = 1 thì hồi tiếp còn tác dụng không?

Trả lời: Khi E = 0, MUX chọn ngõ vào số 0, được nối hồi tiếp từ ngõ ra Q. Do đó, giá trị Q hiện tại được đưa trở lại đầu vào MUX và tiếp tục xuất hiện ở ngõ ra. Nhờ đường hồi tiếp này, mạch duy trì được giá trị đã lưu ngay cả khi tín hiệu D thay đổi.
Khi E = 1, MUX chọn ngõ vào số 1 là tín hiệu D. Khi đó, ngõ ra Q nhận trực tiếp giá trị D và thay đổi theo D. Đường hồi tiếp vẫn tồn tại về mặt vật lý nhưng không được MUX lựa chọn nên không còn quyết định giá trị Q. Vì vậy, D latch có thể vừa lưu giữ dữ liệu khi E = 0, vừa cập nhật dữ liệu khi E = 1.

B. Latch và flip-flop: cùng ngõ vào, khác kết quả

DỰNG MẠCH **Bước 3.2.** Trong circuit **TN3**: Input Pin **D** và **CLK** (đổi bằng Poke, **không** dùng Clock tự chạy); một DLATCH với E = CLK (ngõ ra **Q1**); một **D Flip-Flop** (Memory, kích cạnh lên – Rising Edge) với ngõ vào D, CLK (ngõ ra **Q2**).

DỰ ĐOÁN Bảng 10 cho CLK và D theo từng **khe thời gian** (mỗi cột một khe; tại ranh giới giữa hai khe chỉ đổi **một** tín hiệu: CLK hoặc D, không bao giờ cả hai). Giả sử ban đầu Q1 = Q2 = 0. Điền hàng dự đoán Q1, Q2. Đây là bài "hoàn thành dạng sóng" ở slide Chương 4.

QUAN SÁT Đưa mạch về Q1 = Q2 = 0 (Reset Simulation; D = 0; CLK 0 → 1 → 0). Sau đó Poke từng khe theo bảng và ghi Q1, Q2 đo được. Hàng Q3 để trống đến Bước 3.3.

Bảng 10. Dạng sóng theo khe thời gian

Khe	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CLK	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
D	0	1	1	0	0	1	1	1	1	0	0	1	1	0	0	0
Q1 dự đoán	0	0	1	0	0	0	1	1	1	1	0	1	1	1	0	0
Q1 đo	0	0	1	0	0	0	1	1	1	1	0	1	1	1	0	0
Q2 dự đoán	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0
Q2 đo	0	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0
Q3 đo (DFF_MS)																

Câu 3.3 (Phân tích). Liệt kê các khe mà Q1 ≠ Q2. Với từng khe, giải thích bằng định nghĩa "tích cực theo **mức**" (latch) và "tích cực theo **cạnh**" (flip-flop) ở slide. Trong khoảng CLK = 1, D đổi một lần và Q1 đổi theo, còn Q2 thì không: vì sao?

Trả lời: Các khe có Q1 khác Q2 là 3, 4, 5, 11, 12 và 13.
Tại khe 3 và 11, tín hiệu D thay đổi trong khi CLK = 1. Do D latch hoạt động theo mức cao nên Q1 thay đổi theo D. Ngược lại, D flip-flop chỉ nhận dữ liệu tại cạnh lên của CLK nên Q2 vẫn giữ giá trị trước đó.
Tại các khe 4 và 12, CLK chuyển từ 1 xuống 0 nên D latch chuyển sang trạng thái giữ dữ liệu, còn D flip-flop cũng không cập nhật vì không xuất hiện cạnh lên.
Tại các khe 5 và 13, D thay đổi khi CLK = 0 nên cả hai mạch tiếp tục giữ nguyên giá trị trước đó.
Như vậy, D latch hoạt động theo mức, nghĩa là ngõ ra có thể thay đổi trong suốt thời gian CLK = 1. Trong khi đó, D flip-flop hoạt động theo cạnh, chỉ cập nhật dữ liệu tại thời điểm CLK chuyển từ 0 lên 1 và giữ nguyên giữa các cạnh lên. Đây là nguyên nhân tạo ra sự khác biệt giữa Q1 và Q2.

C. Vì sao thanh ghi dùng flip-flop mà không dùng latch?

DỰNG MẠCH **Bước 3.4.** Trong circuit TN3, dựng hai chuỗi hai tầng: **(i)** hai DLATCH nối tiếp (Q của latch 1 vào D của latch 2), cả hai có E = CLK; **(ii)** hai D Flip-Flop nối tiếp cùng CLK. Cả hai chuỗi dùng chung ngõ vào D.

DỰ ĐOÁN Ban đầu mọi Q = 0. Mỗi **xung** là CLK: 0 → 1 → 0, D giữ nguyên trong xung. Điền Q1, Q2 (viết dạng "Q1 Q2") sau mỗi xung, rồi đo.

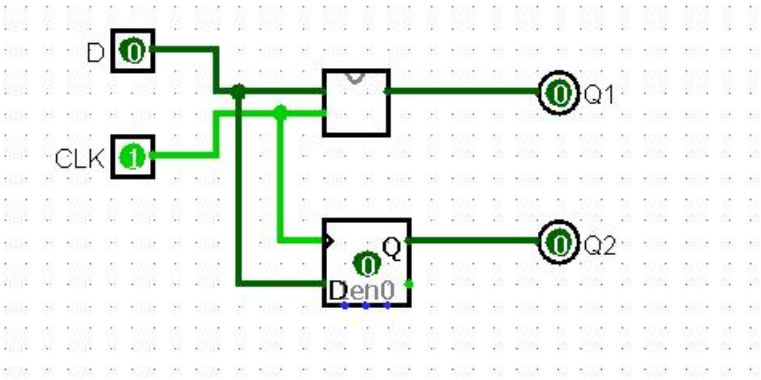
Bảng 12. Chuỗi hai tầng: latch và flip-flop

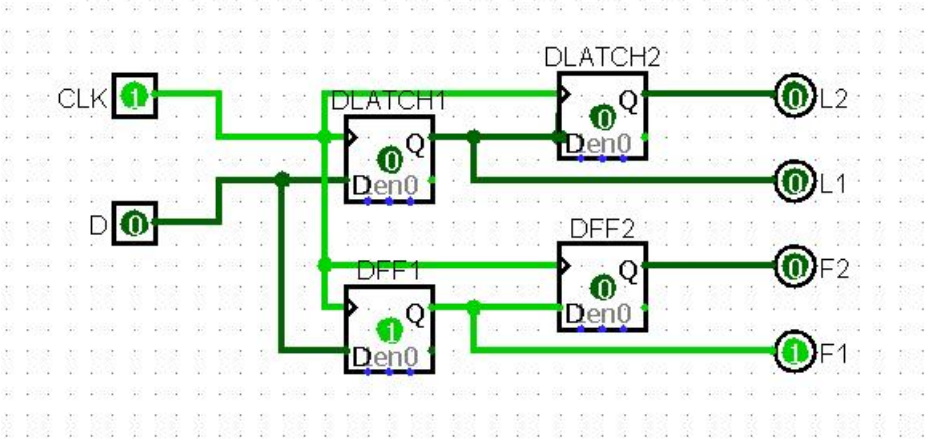
Xung	D	2 latch: dự đoán	2 latch: đo	2 flip-flop: dự đoán	2 flip-flop: đo
1	1	1 1	1 1	1 0	1 0
2	0	0 0	0 0	0 1	0 1
3	0	0 0	0 0	0 0	0 0

Câu 3.5 (Đánh giá). Từ Bảng 12, dữ liệu đi qua mấy tầng trong **một** xung ở mỗi cấu hình? Từ đó dự đoán điều gì xảy ra với lệnh **R1 ← R1 + R0** (ngõ ra ALU quay lại ngõ vào chính thanh ghi R1) khi R1 được làm bằng latch trong suốt lúc CLK = 1, và khi R1 làm bằng flip-flop. (Sinh viên sẽ kiểm chứng ở Thí nghiệm 4.)

Trả lời: Qua Bảng 12, có thể nhận thấy với cấu hình hai D latch nối tiếp dùng chung CLK, khi CLK = 1, cả hai latch đều mở nên dữ liệu có thể truyền qua cả hai tầng trong cùng một xung. Ngược lại, với cấu hình hai D flip-flop nối tiếp, dữ liệu mới chỉ truyền thêm một tầng tại mỗi cạnh lên của CLK do mỗi flip-flop nhận giá trị cũ ở đầu vào trước khi các ngõ ra được cập nhật.
Đối với lệnh R1 ← R1 + R0, nếu thanh ghi R1 được xây dựng bằng latch và trong suốt khi CLK = 1, kết quả của ALU có thể được ghi trở lại R1 ngay trong khoảng CLK đang ở mức cao. Khi R1 thay đổi, ALU tiếp tục tính toán dựa trên giá trị mới, làm kết quả có thể bị cập nhật nhiều lần ngoài ý muốn.
Nếu R1 được xây dựng bằng D flip-flop kích cạnh lên, thanh ghi chỉ cập nhật kết quả tại cạnh lên của CLK và giữ nguyên trong phần còn lại của chu kỳ. Nhờ đó, mỗi cạnh lên chỉ thực hiện một lần cập nhật trạng thái, tránh hiện tượng dữ liệu truyền xuyên qua nhiều tầng hoặc bị tính lặp trong cùng một xung.
Vì vậy, flip-flop phù hợp hơn cho các thanh ghi đồng bộ trong CPU, giúp kiểm soát thời điểm cập nhật dữ liệu và bảo đảm hoạt động ổn định của đường dữ liệu.

Dán ảnh chụp circuit TN3 (Q1, Q2, Q3 và hai chuỗi hai tầng); nếu có, kèm ảnh Chronogram / Logging



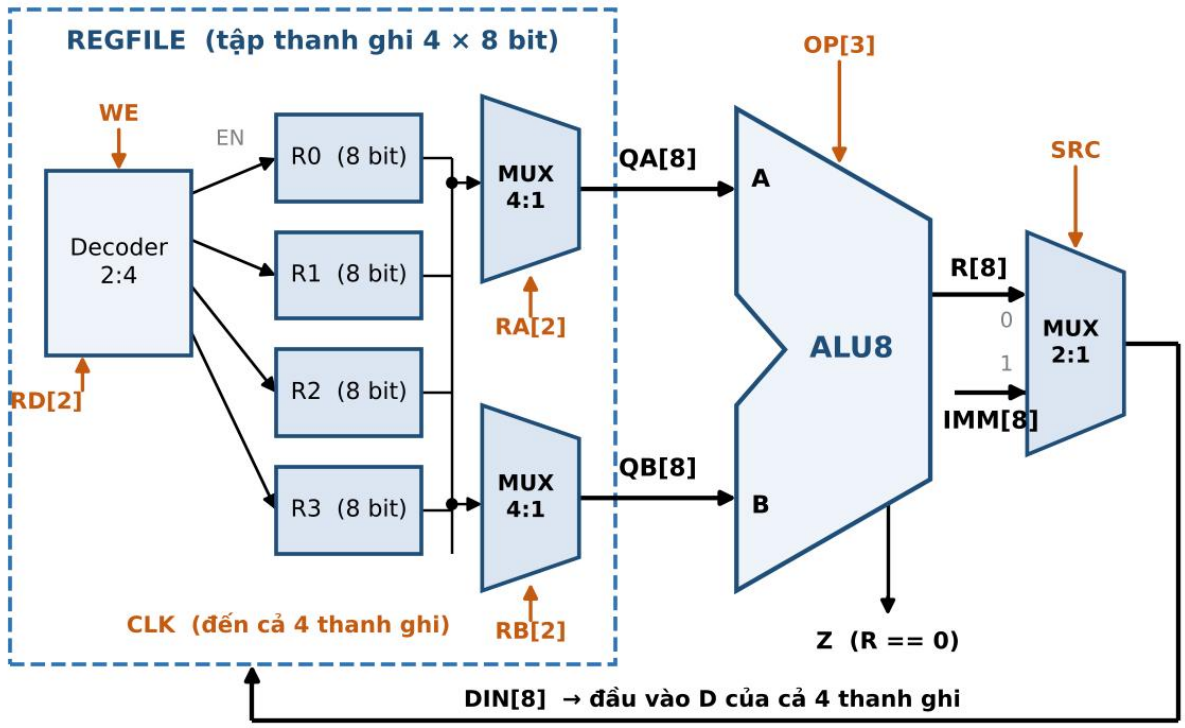


THÍ NGHIỆM 4 — Tập thanh ghi và đường dữ liệu 8 bit

Thời lượng gợi ý: 60 phút · Circuit: **REGFILE, DATAPATH** · Điểm: 3,0

Tình huống

Slide Chương 5 minh hoạ lệnh **C = A + B**: đưa A, B từ bộ nhớ vào **tập thanh ghi**, **ALU** tính, kết quả ghi lại vào tập thanh ghi. Sinh viên ghép các khối đã dựng thành **đường dữ liệu** của một CPU thu nhỏ và điều khiển nó bằng tay: mỗi "lệnh" là một bộ tín hiệu điều khiển cộng với một xung CLK.



Hình 3. *Đường dữ liệu mini: REGFILE (decoder 2:4 + 4 thanh ghi + 2 MUX đọc) → ALU8 → MUX chọn nguồn ghi (SRC).*

A. Tập thanh ghi 4 × 8 bit

Slide Chương 5 vẽ tập thanh ghi với một cổng đọc. ALU cần **hai toán hạng cùng lúc** nên REGFILE ở đây có **hai cổng đọc** (RA, RB) và **một cổng ghi** (RD).

DỰNG MẠCH **Bước 4.1.** Trong circuit **REGFILE**: Input Pin **CLK, WE** (1 bit), **RD** (2 bit, địa chỉ ghi), **DIN** (8 bit), **RA, RB** (2 bit, địa chỉ đọc); Output Pin **QA, QB** (8 bit) và **R0 … R3** (8 bit, để theo dõi). Dùng: một **Decoder 2:4** (Select = RD; nối WE vào ngõ Enable nếu có, hoặc dùng 4 cổng AND với WE), **4 Register** 8 bit (D = DIN, En = ngõ ra thứ k của decoder, Clock = CLK) và **2 Multiplexer 4:1** 8 bit chọn Rk theo RA và theo RB.

DỰ ĐOÁN Với các bước 5–8 của Bảng 13, điền cột "R1 dự đoán" **trước** khi Poke. Mỗi bước kết thúc bằng một xung CLK (0 → 1 → 0) trừ khi ghi chú khác.

QUAN SÁT Ghi R0 … R3 (hex) sau từng bước.

Bảng 13. Ghi và đọc trên REGFILE

Bước	Thao tác	R1 dự đoán	R0	R1	R2	R3
0	Vừa Reset Simulation (WE = 0)	00	00	00	00	00
1	RD = 0, WE = 1, DIN = 0x11, xung CLK	–	11	00	00	00
2	RD = 1, WE = 1, DIN = 0x22, xung CLK	–	11	22	00	00
3	RD = 2, WE = 1, DIN = 0x33, xung CLK	–	11	22	33	00
4	RD = 3, WE = 1, DIN = 0x44, xung CLK	–	11	22	33	44
5	RD = 1, **WE = 0**, DIN = 0x99, xung CLK	22	11	22	33	44
6	RD = 1, WE = 1, DIN = 0x99, **CLK vẫn = 0**	22	11	22	33	44
7	CLK: 0 → 1 (giữ CLK = 1)	99	11	99	33	44
8	CLK: 1 → 0	99	11	99	33	44

Câu 4.1 (Hiểu). Vì sao cần tín hiệu WE ngoài CLK? Điều gì xảy ra nếu bỏ WE (mỗi xung CLK đều ghi DIN vào thanh ghi do RD chọn) khi ta chỉ muốn **so sánh** hai thanh ghi mà không muốn ghi kết quả?

Trả lời: Tín hiệu WE (Write Enable) dùng để quyết định có cho phép ghi dữ liệu vào thanh ghi được chọn hay không, còn CLK quyết định thời điểm cập nhật dữ liệu. Khi WE = 1, thanh ghi được chọn bởi RD sẽ nhận giá trị DIN tại cạnh lên của CLK. Khi WE = 0, thanh ghi giữ nguyên giá trị dù có xung CLK.
Nếu bỏ WE, mỗi cạnh lên của CLK đều ghi DIN vào thanh ghi được chọn bởi RD, kể cả khi chỉ muốn thực hiện phép so sánh mà không cần ghi kết quả. Điều này có thể làm thay đổi dữ liệu đang lưu ngoài ý muốn. Vì vậy, WE giúp kiểm soát thao tác ghi và bảo vệ dữ liệu trong các thanh ghi.

Câu 4.2 (Phân tích). Khi RD = RA = 2, WE = 1 và DIN thay đổi: QA đổi **ngay khi DIN đổi** hay **chỉ sau cạnh lên**? Vì sao điều này quan trọng cho lệnh R2 ← R2 + R1 (liên hệ Bảng 12)?

Trả lời: Khi RD = RA = 2, ngõ ra QA đang đọc dữ liệu từ thanh ghi R2. Nếu WE = 1 và DIN thay đổi nhưng chưa có cạnh lên của CLK, R2 vẫn giữ giá trị cũ nên QA không thay đổi theo DIN. Chỉ khi xuất hiện cạnh lên, R2 mới nhận dữ liệu mới và QA cập nhật theo giá trị mới của R2.
Điều này quan trọng đối với lệnh R2 ← R2 + R1 vì kết quả ALU có thể thay đổi khi R2 được cập nhật. Tuy nhiên, Register chỉ ghi dữ liệu tại cạnh lên của CLK, nên giá trị mới không bị ghi lặp nhiều lần trong cùng một chu kỳ. Nhờ đó, vòng hồi tiếp từ thanh ghi qua ALU trở lại thanh ghi vẫn hoạt động đúng.

B. Ghép đường dữ liệu

DỰNG MẠCH **Bước 4.2.** Trong circuit **DATAPATH**: dùng mạch con REGFILE, mạch con ALU8 và một Multiplexer 2:1 (8 bit) chọn DIN: **SRC = 0 → R** (kết quả ALU), **SRC = 1 → IMM** (hằng số). Input Pin: **IMM** (8 bit), **SRC**, **OP** (3 bit), **RA, RB, RD** (2 bit), **WE**, **CLK**; Output Pin: **R0 … R3**, **R**, **Z**. Nối theo Hình 3 (A ← QA, B ← QB).

Quy ước "một lệnh"

- Một lệnh = đặt đúng các tín hiệu điều khiển (SRC, OP, RA, RB, RD, WE, IMM) **rồi** tạo một xung CLK (0 → 1 → 0).
- Lệnh **nạp hằng**: SRC = 1. Lệnh **ALU**: SRC = 0. Lệnh **chỉ kiểm tra** (không ghi): WE = 0, đọc cờ Z.
- Ô không dùng đến ghi dấu "–".

DỰ ĐOÁN Điền **từ vựng điều khiển** của 5 lệnh ở Bảng 14 (số thanh ghi viết dạng thập phân, OP dạng nhị phân, IMM dạng hex).

Bảng 14. Tín hiệu điều khiển của từng lệnh

Lệnh	SRC	OP	RA	RB	RD	WE	IMM
(1) R0 ← 0x05	1	-	-	-	0	1	0x05
(2) R1 ← 0x07	1	-	-	-	1	1	0x07
(3) R2 ← R0 + R1 (C = A + B ở slide)	0	000	0	1	2	1	-
(4) R3 ← R2 − R0	0	010	2	0	3	1	-
(5) So sánh R3 với R1 (không ghi)	0	010	3	1	-	0	-

QUAN SÁT Chạy lần lượt 5 lệnh (từ trạng thái Reset Simulation) và ghi R0 … R3 (hex) và cờ Z sau mỗi lệnh.

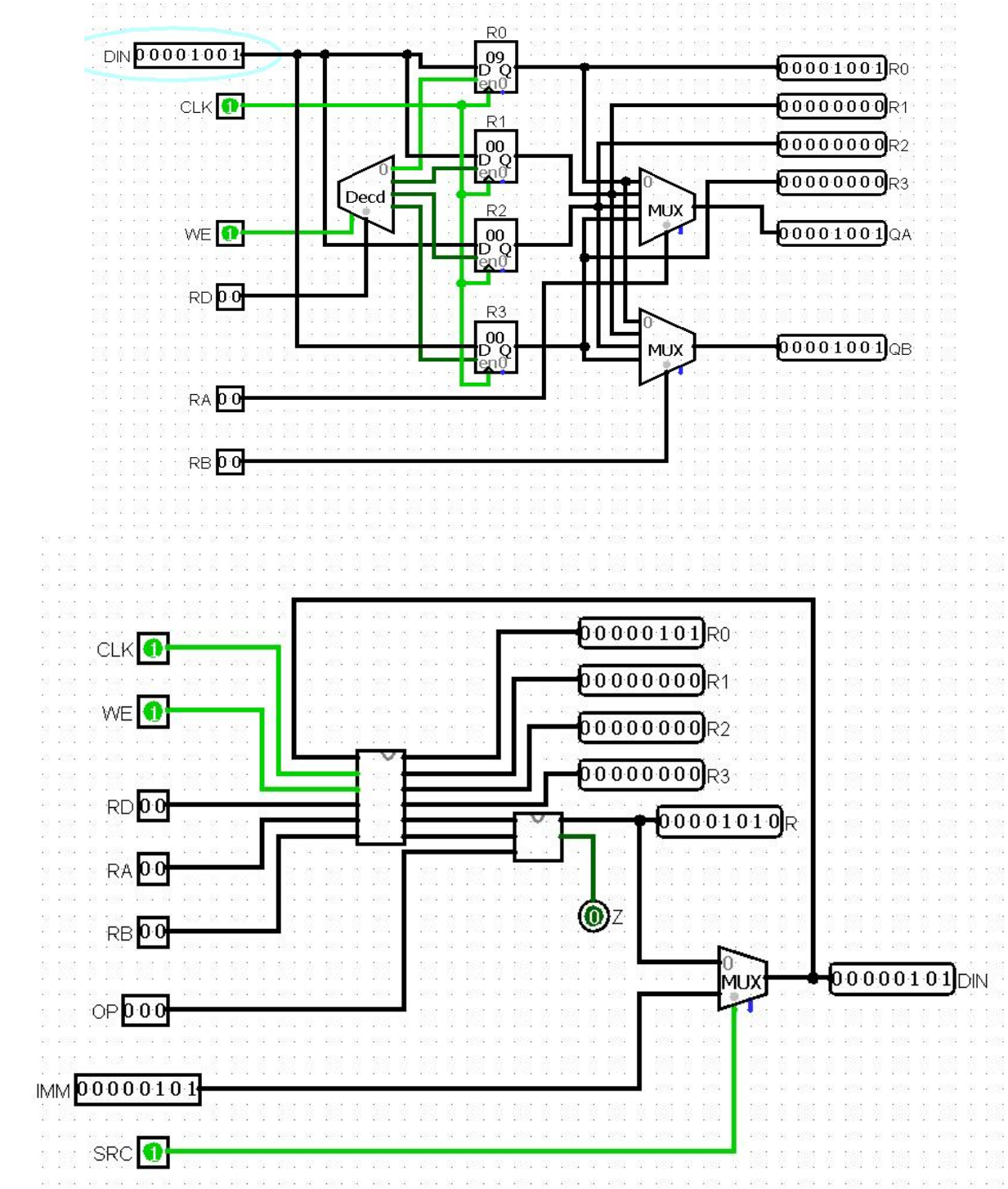
Bảng 15. Kết quả chạy chương trình (1)–(5)

Sau lệnh	R0	R1	R2	R3	Z
(1)	0x05	00	00	00	–
(2)	0x05	0x07	00	00	–
(3)	0x05	0x07	0x0C	00	0
(4)	0x05	0x07	0x0C	0x07	0
(5)	0x05	0x07	0x0C	0x07	1

Câu 4.3 (Hiểu). Lệnh (5) không ghi gì nhưng vẫn hữu ích. Z = 1 cho biết điều gì? So với quy tắc "MSB" ở Thí nghiệm 2, kết luận "bằng nhau" từ Z có luôn đúng không? Vì sao?

Trả lời: Trong lệnh (5), ALU thực hiện phép trừ R3 − R1 mà không ghi kết quả vào thanh ghi do WE = 0. Khi Z = 1, kết quả phép trừ bằng 0, chứng tỏ R3 = R1.
Đối với phép trừ hai toán hạng 8 bit, điều kiện Z = 1 luôn cho biết hai toán hạng bằng nhau, kể cả khi xét số có dấu hoặc không dấu. Điều này khác với quy tắc dựa vào bit MSB để xác định quan hệ lớn hơn hoặc nhỏ hơn, vì quy tắc MSB có thể sai khi phép trừ gây tràn số có dấu hoặc khi các toán hạng được hiểu là số không dấu.
Tuy nhiên, nếu ALU đang thực hiện phép toán khác như AND hoặc OR, Z = 1 chỉ cho biết kết quả phép toán bằng 0 chứ không nhất thiết hai toán hạng bằng nhau.

Dán ảnh chụp circuit TN4



TỔNG KẾT VÀ NỘP BÀI

Danh sách kiểm tra trước khi nộp

☑	Nội dung
☑	File **IT012_Lab02_<MSSV>.circ** có đủ các circuit **FA, ADDSUB4, ALU8, LT, DLATCH, DFF_MS, TN3, REGFILE, DATAPATH**; mọi Input/Output Pin đều có nhãn.
☑	File báo cáo này đã điền đầy đủ các bảng, câu hỏi và dán ảnh chụp mạch vào các khung nét đứt.
☑	Số liệu trong báo cáo khớp với mạch nộp; sau khi sửa mạch con (FA, ALU8…) đã kiểm tra lại các mạch dùng nó.
☑	Đặt tên file báo cáo: **IT012_Lab02_<MSSV>_BaoCao**; nộp cả hai file theo hướng dẫn của giảng viên.

Phản tư

Câu R1. Từ Thí nghiệm 1 và 2, Sinh viên rút ra điều gì về sự khác nhau giữa "kết quả **đúng về mặt bit**" và "kết quả **đúng về mặt ý nghĩa**" (có dấu / không dấu)? Phần cứng giúp phần mềm phân biệt hai điều đó bằng cách nào?

Trả lời: Kết quả đúng về mặt bit là kết quả mà mạch số tính toán chính xác theo độ rộng bit quy định. Tuy nhiên, kết quả đúng về mặt ý nghĩa còn phụ thuộc vào cách diễn giải dữ liệu là số có dấu hay không dấu. Ví dụ, Phép cộng 8 bit 0x7F + 0x01 cho kết quả 0x80, tương ứng với 128 nếu hiểu là số không dấu nhưng là −128 nếu hiểu theo số có dấu bù 2. Do đó, kết quả đúng về mặt bit nhưng phép cộng đã gây tràn số có dấu. Phần cứng hỗ trợ phát hiện các trường hợp này thông qua các cờ trạng thái như Z (Zero), N (Negative), C (Carry) và V (Overflow). Phần mềm dựa vào các cờ và kiểu dữ liệu để diễn giải kết quả phù hợp.

Câu R2. Từ Thí nghiệm 3 và 4: trong DATAPATH, khối nào là mạch **tổ hợp**, khối nào là mạch **tuần tự**? Chỉ ra một vòng hồi tiếp (từ ngõ ra thanh ghi, qua ALU, quay về ngõ vào thanh ghi) và giải thích vì sao vòng này không gây hỗn loạn.

Trả lời: Trong DATAPATH, các khối ALU8, MUX và Decoder thuộc mạch tổ hợp vì ngõ ra phụ thuộc vào giá trị ngõ vào hiện tại. Ngược lại, các Register trong REGFILE thuộc mạch tuần tự vì có khả năng lưu trữ trạng thái và chỉ cập nhật dữ liệu tại cạnh lên của CLK khi tín hiệu WE cho phép ghi.
Một vòng hồi tiếp trong DATAPATH là: ngõ ra thanh ghi → ALU8 → MUX → ngõ vào DIN của REGFILE. Vòng hồi tiếp này không gây ra hiện tượng cập nhật liên tục vì các thanh ghi sử dụng flip-flop kích cạnh lên, chỉ nhận kết quả tại thời điểm có cạnh lên CLK và được cho phép ghi. Trong thời gian còn lại, giá trị thanh ghi được giữ nguyên, giúp quá trình tính toán diễn ra ổn định và đồng bộ.

Câu R3. Nêu **một** lỗi hoặc nhầm lẫn Sinh viên đã gặp trong buổi thực hành, Sinh viên phát hiện ra bằng cách nào và bài học rút ra là gì?

Trả lời: Trong quá trình dựng DATAPATH trên Logisim, em gặp lỗi khi nối tín hiệu SRC 1 bit vào chân Select của MUX đang được cấu hình 2 bit, khiến dây nối hiển thị màu cam và mạch không hoạt động đúng như mong muốn. Em phát hiện lỗi bằng cách quan sát màu dây, kiểm tra độ rộng bit tại các chân kết nối và đối chiếu với yêu cầu của bài thực hành. Sau đó, em điều chỉnh thuộc tính Select Bits của MUX từ 2 xuống 1 và kiểm tra lại hoạt động của mạch. Qua đó, em rút ra bài học rằng khi thiết kế mạch số cần kiểm tra kỹ thuộc tính linh kiện, độ rộng tín hiệu và các kết nối trước khi tiến hành mô phỏng.

Câu R4 (mở, không chấm). Để DATAPATH chạy được một chương trình có vòng lặp hay rẽ nhánh ("nếu Z = 1 thì làm A, ngược lại làm B") mà **không cần người đặt tín hiệu bằng tay**, Sinh viên cần thêm những khối nào? Ghi ý tưởng, chưa cần vẽ mạch.

Trả lời: Để DATAPATH có thể tự động thực hiện chương trình có vòng lặp hoặc rẽ nhánh mà không cần điều khiển bằng tay, cần bổ sung bộ đếm chương trình (PC), bộ nhớ lệnh, bộ giải mã lệnh và bộ điều khiển (Control Unit). Bộ điều khiển sẽ giải mã lệnh và tự động tạo các tín hiệu như SRC, OP, RA, RB, RD và WE. Ngoài ra, cần có mạch kiểm tra các cờ trạng thái như Z và MUX chọn địa chỉ lệnh tiếp theo. Khi gặp lệnh rẽ nhánh, bộ điều khiển dựa vào điều kiện Z để quyết định tiếp tục thực hiện lệnh kế tiếp hay chuyển đến địa chỉ khác. Nhờ đó, CPU có thể tự động thực hiện các lệnh tuần tự, vòng lặp và rẽ nhánh.

Tài liệu tham khảo

- Slide bài giảng Chương 4 – Mạch số và Chương 5 – Ứng dụng mạch số (IT012, Khoa Kỹ thuật Máy tính, UIT).
- Slide bài giảng Chương 2 – Biểu diễn thông tin trong máy tính (số có dấu bù 2) và Chương 3 – Đại số Boole (bìa Karnaugh, chi phí C(B)).
- Tài liệu *Hướng dẫn sử dụng Logisim* do giảng viên cung cấp.